Java OOP Study Notes: Inheritance

1. What is Inheritance?

Inheritance is a mechanism where a new class (child/subclass) acquires the properties and behaviors of an existing class (parent/superclass).

Keyword: `extends`

```
class Parent {
    // Parent class members
}

class Child extends Parent {
    // Child inherits Parent's members
    // Can add its own members too
}
```

Real-World Analogy

```
        Animal
       /      \
     Dog      Cat
    /   \
Labrador German Shepherd
```

A Labrador **is-a** Dog, and a Dog **is-a** Animal. Labrador inherits characteristics from both.

2. Advantages of Inheritance

Advantage	Description
Code Reusability	Write once in parent, use in all children
Extensibility	Add new features without modifying existing code
Maintainability	Fix bugs in one place (parent), affects all children
Polymorphism	Treat different objects uniformly through parent type
Logical Structure	Models real-world relationships (is-a)

Without Inheritance (Code Duplication)

```
class Dog {
    String name;
    void eat() { System.out.println("Eating..."); }
    void sleep() { System.out.println("Sleeping..."); }
    void bark() { System.out.println("Barking!"); }
}

class Cat {
    String name;
    void eat() { System.out.println("Eating..."); }    // Duplicated!
    void sleep() { System.out.println("Sleeping..."); } // Duplicated!
    void meow() { System.out.println("Meowing!"); }
}
```

With Inheritance (DRY - Don't Repeat Yourself)

```
class Animal {
    String name;
    void eat() { System.out.println("Eating..."); }
    void sleep() { System.out.println("Sleeping..."); }
}

class Dog extends Animal {
    void bark() { System.out.println("Barking!"); }
}

class Cat extends Animal {
    void meow() { System.out.println("Meowing!"); }
}
```

---

3. Types of Inheritance

3.1 Single Inheritance

One child class inherits from one parent class.

```
    Parent
       |
       ▼
     Child
```

```
class Vehicle {
    String brand;

    void start() {
        System.out.println("Vehicle starting...");
    }
}

class Car extends Vehicle {
    int numDoors;

    void drive() {
        System.out.println("Car driving...");
    }
}

// Usage
Car car = new Car();
car.brand = "Toyota";  // Inherited from Vehicle
car.numDoors = 4;      // Own property
car.start();           // Inherited method
car.drive();           // Own method
```

3.2 Multilevel Inheritance

A class inherits from a class that also inherits from another class (grandparent → parent → child).

```
  Grandparent
       |
       ▼
    Parent
       |
       ▼
     Child
```

```
class Animal {
    void eat() {
        System.out.println("Animal eating");
    }
}

class Mammal extends Animal {
    void breathe() {
        System.out.println("Mammal breathing");
    }
}

class Dog extends Mammal {
    void bark() {
        System.out.println("Dog barking");
    }
}

// Usage
Dog dog = new Dog();
dog.eat();      // From Animal (grandparent)
dog.breathe();  // From Mammal (parent)
dog.bark();     // Own method
```

Inheritance Chain

```
// Dog inherits from Mammal, which inherits from Animal
// Dog has access to ALL methods up the chain

class Animal {
    void eat() { }
}
         ↑
class Mammal extends Animal {
    void breathe() { }
}
         ↑
class Dog extends Mammal {
    void bark() { }
}
         ↑
class Labrador extends Dog {
    void fetch() { }
    // Can call: eat(), breathe(), bark(), fetch()
}
```

3.3 Hierarchical Inheritance

Multiple child classes inherit from a single parent class.

```
         Parent
        /  |  \
       ▼   ▼   ▼
   Child1 Child2 Child3
```

```
class Shape {
    String color;

    void draw() {
        System.out.println("Drawing shape");
    }
}

class Circle extends Shape {
    double radius;

    double getArea() {
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {
    double width, height;

    double getArea() {
        return width * height;
    }
}

class Triangle extends Shape {
    double base, height;

    double getArea() {
        return 0.5 * base * height;
    }
}

// Usage
Circle c = new Circle();
c.color = "Red";     // Inherited
c.radius = 5;
c.draw();            // Inherited

Rectangle r = new Rectangle();
r.color = "Blue";    // Same inherited property
```

```
r.width = 10;
r.height = 5;
r.draw();            // Same inherited method
```

3.4 Multiple Inheritance (NOT Supported Directly)

Java does **NOT** support multiple inheritance with classes (a class cannot extend more than one class).

```
   Parent1    Parent2
       \        /
        \      /
         \    /
          ▼  ▼
         Child     ← NOT ALLOWED in Java!
```

```
class A { }
class B { }

// class C extends A, B { }  // COMPILE ERROR!
```

Why Not?

Diamond Problem: If both parents have the same method, which one should the child inherit?

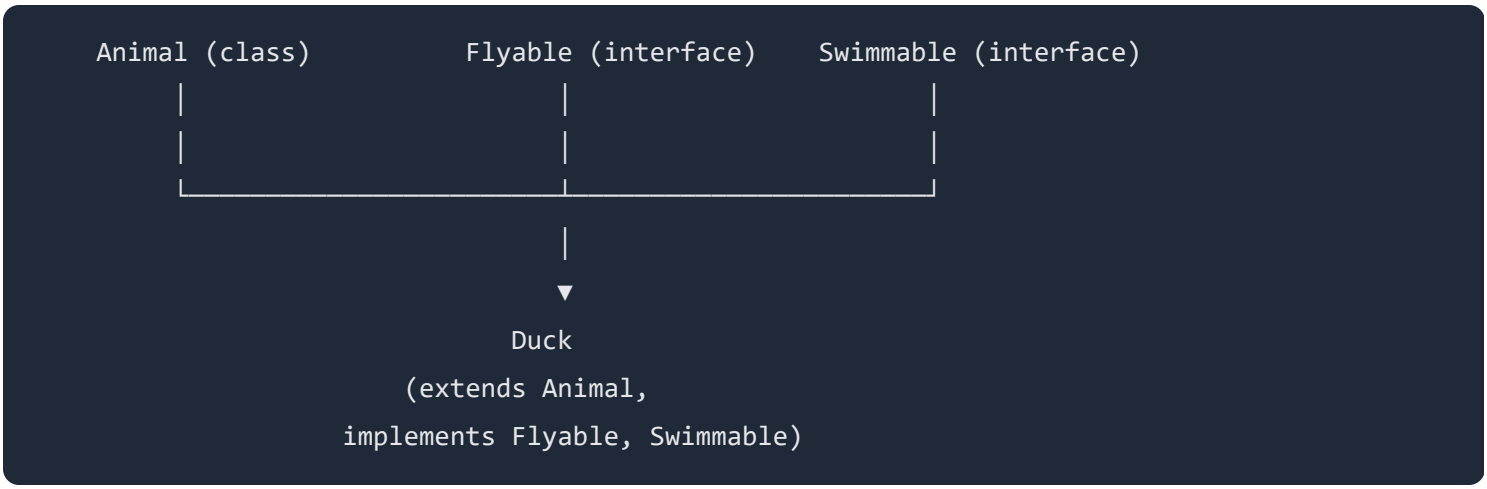
```
        Animal
       /      \
    Flyer    Swimmer
       \      /
        \    /
         Duck   ← Which eat() method? Ambiguous!
```

3.5 Hybrid Inheritance (via Interfaces)

Java achieves multiple inheritance through **interfaces**.

```
interface Flyable {
    void fly();
}

interface Swimmable {
    void swim();
}

class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

// Duck extends ONE class but implements MULTIPLE interfaces
class Duck extends Animal implements Flyable, Swimmable {

    @Override
    public void fly() {
        System.out.println("Duck flying");
    }

    @Override
    public void swim() {
        System.out.println("Duck swimming");
    }
}

// Usage
Duck duck = new Duck();
duck.eat();   // From Animal class
duck.fly();   // From Flyable interface
duck.swim();  // From Swimmable interface
```

Hybrid Inheritance Diagram



---

4. The `super` Keyword

The `super` keyword refers to the **parent class**. It's used to:

1. Call parent constructor
2. Access parent methods
3. Access parent variables

4.1 Access Parent Constructor: `super()`

```
class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
        System.out.println("Person constructor called");
    }
}

class Student extends Person {
    String studentId;

    Student(String name, int age, String studentId) {
        super(name, age);  // MUST be first line! Calls Person constructor
        this.studentId = studentId;
        System.out.println("Student constructor called");
    }
}

// Usage
Student s = new Student("Aslam", 20, "STU001");

// Output:
// Person constructor called
// Student constructor called
```

Rules for `super()`

Rule	Description
Must be **first statement**	No code can come before `super()`
Implicit `super()`	If you don't write it, Java adds `super()` (no-arg)
Must match parent constructor	Arguments must match a parent constructor signature

```
class Parent {
    Parent(int x) {  // Only parameterized constructor
        System.out.println("Parent: " + x);
    }
}

class Child extends Parent {
    Child() {
        // super();  // ERROR! Parent has no no-arg constructor
        super(10);   // Must call the existing constructor
    }
}
```

4.2 Access Parent Methods: `super.method()`

When child overrides a method but still needs parent's version:

```
class Animal {
    void makeSound() {
        System.out.println("Some generic animal sound");
    }

    void eat() {
        System.out.println("Animal eating");
    }
}

class Dog extends Animal {

    @Override
    void makeSound() {
        System.out.println("Bark! Bark!");
    }

    void communicate() {
        super.makeSound();  // Calls Animal's makeSound()
        makeSound();        // Calls Dog's makeSound()
    }

    @Override
    void eat() {
        super.eat();        // First do what Animal does
        System.out.println("Dog eating dog food");  // Then add more
    }
}

// Usage
Dog dog = new Dog();
dog.communicate();
// Output:
// Some generic animal sound
// Bark! Bark!

dog.eat();
// Output:
```

```
// Animal eating
// Dog eating dog food
```

4.3 Access Parent Variables: `super.variable`

When child has a variable with the same name as parent:

```
class Parent {
    String message = "Hello from Parent";
    int value = 100;
}

class Child extends Parent {
    String message = "Hello from Child";  // Same name - shadows parent
    int value = 200;

    void showMessages() {
        System.out.println(message);        // Child's message
        System.out.println(super.message);  // Parent's message

        System.out.println(value);          // 200 (Child's)
        System.out.println(super.value);    // 100 (Parent's)
    }
}

// Usage
Child c = new Child();
c.showMessages();

// Output:
// Hello from Child
// Hello from Parent
// 200
// 100
```

4.4 `super` vs `this` Comparison

Keyword	Refers To	Constructor Call	Variable/Method Access
`this`	Current object	`this()` - another constructor in same class	`this.variable`, `this.method()`
`super`	Parent object	`super()` - parent constructor	`super.variable`, `super.method()`

```
class Parent {
    int x = 10;
    void show() { System.out.println("Parent show"); }
}

class Child extends Parent {
    int x = 20;

    void show() {
        System.out.println("Child show");
    }

    void display() {
        System.out.println(x);         // 20 (Child's x)
        System.out.println(this.x);    // 20 (same as above)
        System.out.println(super.x);   // 10 (Parent's x)

        show();         // Child's show()
        this.show();    // Child's show()
        super.show();   // Parent's show()
    }
}
```

---

5. Method Overriding (Runtime Polymorphism)

Method Overriding occurs when a child class provides a specific implementation for a method already defined in its parent class.

Rules for Overriding

Rule	Description
Same method name	Must match exactly
Same parameters	Same number, type, and order
Same or covariant return type	Can return subtype of parent's return type
Cannot reduce visibility	If parent is `public`, child cannot be `private`
Cannot override `final` methods	Final methods cannot be overridden
Cannot override `static` methods	Static methods are hidden, not overridden
Use `@Override` annotation	Recommended for compiler checking

Basic Example

```
class Animal {
    void makeSound() {
        System.out.println("Some sound...");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Bark! Bark!");
    }
}

class Cat extends Animal {
    @Override
    void makeSound() {
        System.out.println("Meow!");
    }
}

class Cow extends Animal {
    @Override
    void makeSound() {
        System.out.println("Moo!");
    }
}
```

Runtime Polymorphism (Dynamic Method Dispatch)

The method to call is decided at **runtime** based on the actual object type, not the reference type.

```
public class Main {
    public static void main(String[] args) {

        // Parent reference, different child objects
        Animal a1 = new Dog();
        Animal a2 = new Cat();
        Animal a3 = new Cow();

        a1.makeSound();  // Output: Bark! Bark!
        a2.makeSound();  // Output: Meow!
        a3.makeSound();  // Output: Moo!

        // Powerful: Process different animals uniformly
        Animal[] animals = { new Dog(), new Cat(), new Cow(), new Dog() };

        for (Animal animal : animals) {
            animal.makeSound();  // Correct method called for each type
        }
    }
}
```

Output:

```
Bark! Bark!
Meow!
Moo!
Bark! Bark!
Meow!
Moo!
Bark! Bark!
```

How It Works

```
Compile Time                    Runtime
____________________            ____________________

Animal a = new Dog();           JVM sees actual object is Dog
a.makeSound();                  Calls Dog's makeSound()
     |                               |
     ▼                               ▼
Compiler checks:                JVM executes:
"Does Animal have               Dog.makeSound()
 makeSound()?" ✓                → "Bark! Bark!"
```

Covariant Return Types

Child can return a **subtype** of parent's return type:

```
class Animal {
    Animal reproduce() {
        return new Animal();
    }
}

class Dog extends Animal {
    @Override
    Dog reproduce() {  // Returns Dog (subtype of Animal) - VALID!
        return new Dog();
    }
}
```

Access Modifier Rules

```
class Parent {
    protected void show() { }
}

class Child extends Parent {
    // @Override
    // private void show() { }  // ERROR! Cannot reduce visibility

    @Override
    public void show() { }      // OK! Can increase visibility
}
```

Parent Modifier	Child Can Use
`public`	`public` only
`protected`	`protected` or `public`
`default`	`default`, `protected`, or `public`
`private`	Cannot override (not inherited)

---

6. Method Overriding vs Method Overloading

Feature	Overriding	Overloading
Definition	Same method in child class	Same method name, different parameters
Classes	Requires inheritance (2 classes)	Same class (or inherited)
Parameters	Must be same	Must be different
Return Type	Same or covariant	Can be different
Binding	Runtime (dynamic)	Compile time (static)
Keyword	`@Override` annotation	None

```
class Calculator {
    // OVERLOADING - same class, different parameters
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
}

class Animal {
    void speak() {
        System.out.println("Animal speaks");
    }
}

class Dog extends Animal {
    // OVERRIDING - child class, same parameters
    @Override
    void speak() {
        System.out.println("Dog barks");
    }
}
```

---

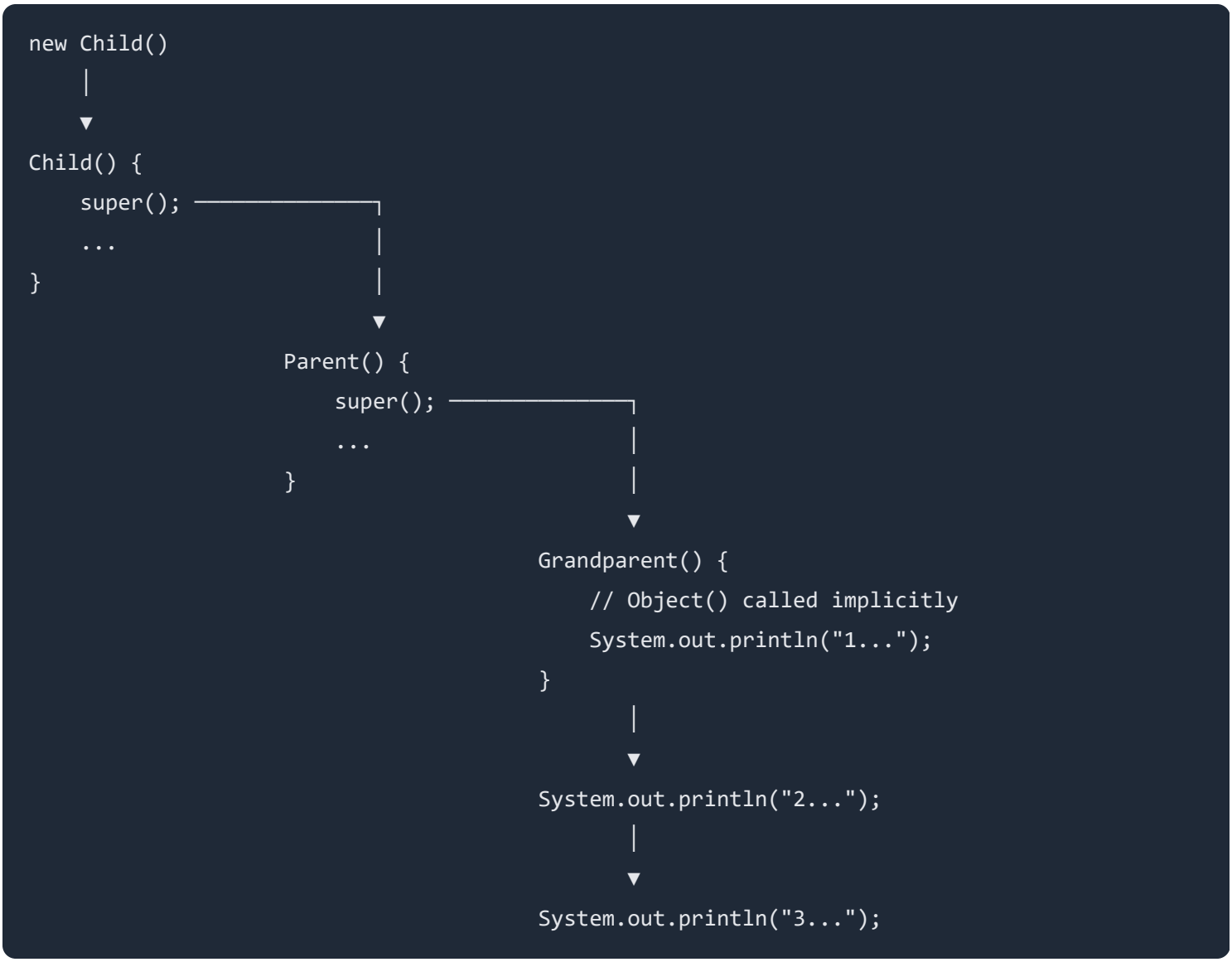
7. Constructor Chaining in Inheritance

When an object is created, constructors execute from **top to bottom** of the inheritance hierarchy.

Automatic Chaining

```
class Grandparent {
    Grandparent() {
        System.out.println("1. Grandparent constructor");
    }
}

class Parent extends Grandparent {
    Parent() {
        // super(); ← Implicit call to Grandparent()
        System.out.println("2. Parent constructor");
    }
}

class Child extends Parent {
    Child() {
        // super(); ← Implicit call to Parent()
        System.out.println("3. Child constructor");
    }
}

// Usage
Child c = new Child();

// Output (top to bottom):
// 1. Grandparent constructor
// 2. Parent constructor
// 3. Child constructor
```

Execution Flow

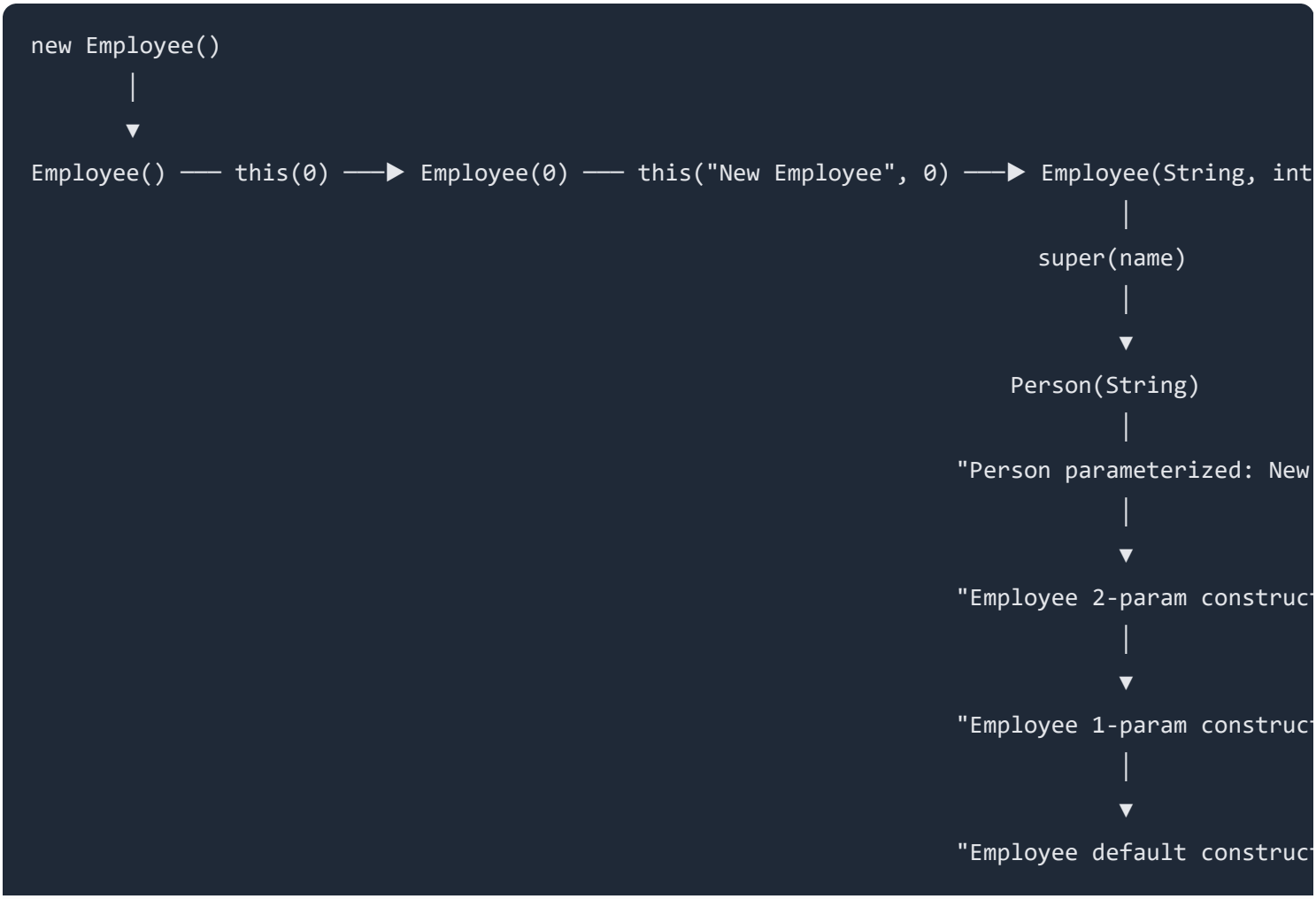


Explicit Constructor Chaining with Parameters

```
class Person {
    String name;

    Person() {
        this("Unknown");
        System.out.println("Person default constructor");
    }

    Person(String name) {
        this.name = name;
        System.out.println("Person parameterized: " + name);
    }
}

class Employee extends Person {
    int empId;

    Employee() {
        this(0);
        System.out.println("Employee default constructor");
    }

    Employee(int empId) {
        this("New Employee", empId);
        System.out.println("Employee 1-param constructor");
    }

    Employee(String name, int empId) {
        super(name);  // Call Person(String)
        this.empId = empId;
        System.out.println("Employee 2-param constructor");
    }
}

// Usage
Employee e = new Employee();

// Output:
```

```
// Person parameterized: New Employee
// Employee 2-param constructor
// Employee 1-param constructor
// Employee default constructor
```

Flow Diagram



8. Complete Inheritance Example

```
// ============================================================
// BASE CLASS
// ============================================================
class Employee {
    protected int id;
    protected String name;
    protected double baseSalary;

    private static int counter = 0;

    Employee(String name, double baseSalary) {
        this.id = ++counter;
        this.name = name;
        this.baseSalary = baseSalary;
        System.out.println("Employee constructor: " + name);
    }

    double calculateSalary() {
        return baseSalary;
    }

    void displayInfo() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Base Salary: $" + baseSalary);
    }

    void work() {
        System.out.println(name + " is working...");
    }
}

// ============================================================
// DEVELOPER - Single Inheritance
// ============================================================
class Developer extends Employee {
    private String programmingLanguage;
    private int projectBonus;
```

```
    Developer(String name, double baseSalary, String language) {
        super(name, baseSalary);
        this.programmingLanguage = language;
        this.projectBonus = 0;
        System.out.println("Developer constructor: " + language);
    }

    void setProjectBonus(int bonus) {
        this.projectBonus = bonus;
    }

    @Override
    double calculateSalary() {
        return super.calculateSalary() + projectBonus;
    }

    @Override
    void displayInfo() {
        super.displayInfo();
        System.out.println("Language: " + programmingLanguage);
        System.out.println("Project Bonus: $" + projectBonus);
        System.out.println("Total Salary: $" + calculateSalary());
    }

    @Override
    void work() {
        System.out.println(name + " is coding in " + programmingLanguage);
    }

    void debug() {
        System.out.println(name + " is debugging code...");
    }
}

// ============================================================
// SENIOR DEVELOPER - Multilevel Inheritance
// ============================================================
class SeniorDeveloper extends Developer {
    private int teamSize;
```

```
    private double leadershipBonus;

    SeniorDeveloper(String name, double baseSalary, String language, int teamSize) {
        super(name, baseSalary, language);
        this.teamSize = teamSize;
        this.leadershipBonus = teamSize * 500;  // $500 per team member
        System.out.println("SeniorDeveloper constructor: Team of " + teamSize);
    }

    @Override
    double calculateSalary() {
        return super.calculateSalary() + leadershipBonus;
    }

    @Override
    void displayInfo() {
        super.displayInfo();
        System.out.println("Team Size: " + teamSize);
        System.out.println("Leadership Bonus: $" + leadershipBonus);
        System.out.println("Final Salary: $" + calculateSalary());
    }

    @Override
    void work() {
        super.work();
        System.out.println(name + " is also leading a team of " + teamSize);
    }

    void conductCodeReview() {
        System.out.println(name + " is conducting code review...");
    }
}

// ============================================================
// MANAGER - Hierarchical Inheritance (also extends Employee)
// ============================================================
class Manager extends Employee {
    private String department;
    private double managerBonus;
```

```
    Manager(String name, double baseSalary, String department) {
        super(name, baseSalary);
        this.department = department;
        this.managerBonus = 10000;
        System.out.println("Manager constructor: " + department);
    }

    @Override
    double calculateSalary() {
        return super.calculateSalary() + managerBonus;
    }

    @Override
    void displayInfo() {
        super.displayInfo();
        System.out.println("Department: " + department);
        System.out.println("Manager Bonus: $" + managerBonus);
        System.out.println("Total Salary: $" + calculateSalary());
    }

    @Override
    void work() {
        System.out.println(name + " is managing the " + department + " department");
    }

    void conductMeeting() {
        System.out.println(name + " is conducting a team meeting...");
    }
}

// ============================================================
// MAIN CLASS - Testing
// ============================================================
public class Main {
    public static void main(String[] args) {
        System.out.println("=======================================");
        System.out.println("Creating Developer:");
        System.out.println("=======================================");
        Developer dev = new Developer("Aslam", 60000, "Java");
        dev.setProjectBonus(5000);
```

```
        System.out.println("\n========================================");
        System.out.println("Creating Senior Developer:");
        System.out.println("========================================");
        SeniorDeveloper seniorDev = new SeniorDeveloper("Rahim", 80000, "Python", 5);
        seniorDev.setProjectBonus(8000);

        System.out.println("\n========================================");
        System.out.println("Creating Manager:");
        System.out.println("========================================");
        Manager manager = new Manager("Karim", 90000, "Engineering");

        // Polymorphism - treat all as Employee
        System.out.println("\n========================================");
        System.out.println("Polymorphism Demo:");
        System.out.println("========================================");

        Employee[] employees = { dev, seniorDev, manager };

        for (Employee emp : employees) {
            System.out.println("\n--- " + emp.name + " ---");
            emp.work();  // Calls overridden method
            System.out.println("Calculated Salary: $" + emp.calculateSalary());
        }

        // Full info display
        System.out.println("\n========================================");
        System.out.println("Detailed Info:");
        System.out.println("========================================");

        System.out.println("\n[Developer]");
        dev.displayInfo();

        System.out.println("\n[Senior Developer]");
        seniorDev.displayInfo();

        System.out.println("\n[Manager]");
        manager.displayInfo();
```

```
    }
}
```

Output

```
==========================================
Creating Developer:
==========================================
Employee constructor: Aslam
Developer constructor: Java

==========================================
Creating Senior Developer:
==========================================
Employee constructor: Rahim
Developer constructor: Python
SeniorDeveloper constructor: Team of 5

==========================================
Creating Manager:
==========================================
Employee constructor: Karim
Manager constructor: Engineering

==========================================
Polymorphism Demo:
==========================================

--- Aslam ---
Aslam is coding in Java
Calculated Salary: $65000.0

--- Rahim ---
Rahim is coding in Python
Rahim is also leading a team of 5
Calculated Salary: $90500.0

--- Karim ---
Karim is managing the Engineering department
Calculated Salary: $100000.0

==========================================
Detailed Info:
```

```
========================================

[Developer]
ID: 1
Name: Aslam
Base Salary: $60000.0
Language: Java
Project Bonus: $5000
Total Salary: $65000.0

[Senior Developer]
ID: 2
Name: Rahim
Base Salary: $80000.0
Language: Python
Project Bonus: $8000
Total Salary: $90500.0
Team Size: 5
Leadership Bonus: $2500.0
Final Salary: $90500.0

[Manager]
ID: 3
Name: Karim
Base Salary: $90000.0
Department: Engineering
Manager Bonus: $10000.0
Total Salary: $100000.0
```

9. Inheritance Hierarchy Diagram

```
classDiagram
    class Employee {
        #int id
        #String name
        #double baseSalary
        +calculateSalary() double
        +displayInfo() void
        +work() void
    }

    class Developer {
        -String programmingLanguage
        -int projectBonus
        +debug() void
    }

    class SeniorDeveloper {
        -int teamSize
        -double leadershipBonus
        +conductCodeReview() void
    }

    class Manager {
        -String department
        -double managerBonus
        +conductMeeting() void
    }

    Employee <|-- Developer : extends
    Employee <|-- Manager : extends
    Developer <|-- SeniorDeveloper : extends
```

---

10. What is NOT Inherited?

NOT Inherited	Reason
Private members	Not accessible outside class
Constructors	Must be called via `super()`, not inherited
Static members	Belong to class, not object (but accessible)
Final methods	Cannot be overridden

```
class Parent {
    private int secret = 42;       // NOT inherited
    public int visible = 100;      // Inherited

    Parent() { }                   // NOT inherited (called via super())

    static void staticMethod() { } // NOT inherited (but accessible via Parent.staticMethod())
}

class Child extends Parent {
    void test() {
        // System.out.println(secret);  // ERROR! private
        System.out.println(visible);     // OK - inherited
    }
}
```

---

11. Key Points to Remember

1. **Inheritance** = Child class acquires parent class properties (`extends`)
2. **Single Inheritance** = One parent, one child
3. **Multilevel** = Chain: Grandparent → Parent → Child
4. **Hierarchical** = One parent, multiple children

5. **Multiple inheritance** NOT allowed with classes (use interfaces)
6. `super()` calls parent constructor (must be first line)
7. `super.method()` calls parent's version of overridden method
8. `super.variable` accesses parent's variable when shadowed
9. **Method Overriding** = Same signature in child (runtime polymorphism)
10. `@Override` annotation recommended for overriding
11. **Constructor chaining** executes top-down in inheritance hierarchy
12. **Private members** are NOT inherited
13. **Constructors** are NOT inherited (but called via `super()`)

'; triggerDownload(docTitle.replace(/[^a-zA-Z0-9\s-]/g, '') + '.html', html, 'text/html;charset=utf-8'); } function downloadPDF() { document.getElementById('downloadDropdown').classList.remove('show'); // Force light mode for PDF (dark backgrounds print poorly) const wasDark = document.documentElement.classList.contains('dark'); if (wasDark) document.documentElement.classList.remove('dark'); const element = document.getElementById('document-content'); const opt = { margin: [0.75, 0.75, 0.75, 0.75], filename: docTitle.replace(/[^a-zA-Z0-9\s-]/g, '') + '.pdf', image: { type: 'jpeg', quality: 0.98 }, html2canvas: { scale: 1.5, useCORS: true, backgroundColor: '#ffffff' }, jsPDF: { unit: 'in', format: 'letter', orientation: 'portrait' }, pagebreak: { mode: ['css', 'legacy'] } }; html2pdf().set(opt).from(element).save().then(function() { if (wasDark) document.documentElement.classList.add('dark'); }); } /* Syntax highlighting and mermaid rendering */ (async function init() { const isDark = window.matchMedia('(prefers-color-scheme: dark)').matches; // Highlight code blocks (skip mermaid) document.querySelectorAll('pre code').forEach(function(block) { const lang = block.className.replace('language-', ''); if (lang !== 'mermaid') { hljs.highlightElement(block); } }); updateHljsTheme(isDark); // Render mermaid diagrams try { mermaid.initialize({ startOnLoad: false, theme: isDark ? 'dark' : 'default', securityLevel: 'loose' }); const mermaidBlocks = document.querySelectorAll('pre code.language-mermaid'); for (let i = 0; i < mermaidBlocks.length; i++) { const block = mermaidBlocks[i]; const pre = block.parentElement; const code = block.textContent; try { const { svg } = await mermaid.render('mermaid-' + i, code); const container = document.createElement('div'); container.className = 'mermaid-container'; container.innerHTML = svg; pre.replaceWith(container); } catch (err) { console.warn('Mermaid render failed:', err); } } } catch (err) { console.warn('Mermaid init failed:', err); } })(); function updateHljsTheme(isDark) { document.getElementById('hljs-dark').disabled = !isDark; document.getElementById('hljs-light').disabled = isDark; } /* Dark mode sync from parent */

window.addEventListener('message', function(e) { if (e.data && e.data.type === 'darkMode') { const isDark = e.data.dark; document.documentElement.classList.toggle('dark', isDark); updateHljsTheme(isDark); // Re-init mermaid theme try { mermaid.initialize({ startOnLoad: false, theme: isDark ? 'dark' : 'default', securityLevel: 'loose' }); } catch (err) {} } });